LiftLog Diet Tracking API & Database Specification

LiftLog - Diet Tracking Module

1. Overview

The LiftLog Diet Tracking module is designed around the same basic experience as HealthifyMe:

- User has a daily calorie and macro target.
- User searches/browses foods.
- Foods have nutritional information per gram and human-friendly serving sizes.
- User can log food into a particular meal.
- User can create custom foods when a food isn't available.
- User can save frequently consumed combinations as meals.
- User can add saved meals to a day.
- User can see calories and macros consumed throughout the day.
- User can edit/delete logged food.
- Historical diet data can be retrieved for analytics and AI coaching.

The module should support:

2. Core Concepts

There are four different concepts that should NOT be mixed together.

Food

A single food item.

Examples:

- Cooked Rice
- Banana

- Chicken Breast
- Egg
- Milk

Serving
A human-friendly quantity of a food.

Examples:

```
Cooked Rice
    100 g
    Small Bowl = 150 g
    Medium Bowl = 200 g
    Large Bowl = 300 g
Banana
    100 g
    Small Banana = 80 g
    Medium Banana = 118 g
    Large Banana = 136 g
```

The food’s nutritional information is ultimately based on grams.

Meal

A reusable combination of foods.

Example:

```
Meal: My Breakfast
2 Eggs
2 slices Bread
1 Banana
250 ml Milk
```

Food Log
The actual food consumed on a particular date.

Example:

```
2026-09-06
Breakfast
    2 Eggs
    2 slices Bread
    1 Banana
Lunch
    250g Rice
```

```
    150g Chicken
    Vegetables
```

This distinction is important because a saved meal is a reusable template, while a food log is historical data.

3. Recommended API Structure

I recommend the following API groups:

```
/api/v1/diet/summary
/api/v1/diet/foods
/api/v1/diet/custom-foods
/api/v1/diet/meals
/api/v1/diet/logs
/api/v1/diet/goals
```

4. API #1 - Daily Nutrition Summary

Endpoint

GET /api/v1/diet/summary

Query Parameters
date

Example:

GET /api/v1/diet/summary?date=2026-09-06

If date is omitted, default to today.

Response


```
{
  "date": "2026-09-06",
  "calories": {
    "consumed": 1850,
    "target": 2500,
    "remaining": 650
  },
  "macros": {
    "protein": {
      "consumed_g": 135,
      "target_g": 170,
      "remaining_g": 35
    },
    "carbs": {
      "consumed_g": 210,
      "target_g": 280,
      "remaining_g": 70
    },
    "fat": {
      "consumed_g": 55,
      "target_g": 75,
```

```
      "remaining_g": 20
    }
  },
  "meals": {
    "breakfast": {
      "calories": 450,
      "protein_g": 50,
      "carbs": 50,
      "fat": 50
    },
    "lunch": {
      "calories": 700,
      "protein_g": 50,
      "carbs": 50,
      "fat": 50
    },
    "snack": {
      "calories": 250,
      "protein_g": 50,
      "carbs": 50,
      "fat": 50
    },
    "dinner": {
      "calories": 450,
      "protein_g": 50,
      "carbs": 50,
      "fat": 50
    }
  }
}
```


5. API #2 - Food Database

Endpoint

```
GET /api/v1/diet/foods
```

This should support search.
Query Parameters

```
search
category
page
page_size
```

Example:

```
GET /api/v1/diet/foods?search=rice
```

Response


```
{
    "items": [
        {
            "food_id": 101,
            "name": "Cooked Rice",
            "nutrition_per_100g": {
                "calories": 130,
                "protein_g": 2.7,
                "carbs_g": 28.2,
                "fat_g": 0.3
            }

        },
        {
            "food_id": 102,
            "name": "Fried Rice",
            "nutrition_per_100g": {
                "calories": 150,
                "protein_g": 1.7,
                "carbs_g": 29.2,
                "fat_g": 0.9
            }

        }
```

```
    ],
    "pagination": {
        "page": 1,
        "page_size": 20,
        "total": 1
    }
}
```

API #3 - GET /api/v1/diet/foods/{food_id}

```
{
            "food_id": 101,
            "name": "Cooked Rice",
            "category": "Grains",
            "nutrition_per_100g": {
                "calories": 130,
                "protein_g": 2.7,
                "carbs_g": 28.2,
                "fat_g": 0.3
            },
            "servings": [
                {
                    "serving_id": 1011,
                    "name": "Small Bowl",
                    "quantity_g": 150,
                    "calories": 195,
                    "protein_g": 4.05,
                    "carbs_g": 42.3,
                    "fat_g": 0.45
                },
                {
                    "serving_id": 1012,
                    "name": "Medium Bowl",
                    "quantity_g": 200,
                    "calories": 260,
                    "protein_g": 5.4,
```

```
                "carbs_g": 56.4,
                "fat_g": 0.6
            },
            {
                "serving_id": 1013,
                "name": "Large Bowl",
                "quantity_g": 300,
                "calories": 390,
                "protein_g": 8.1,
                "carbs_g": 84.6,
                "fat_g": 0.9
            }
        ]
        }
```


6. Food Serving Design

This is an important part of the system.

Do NOT store:

```
small bowl = 195 calories
```

as the primary nutritional information.

Instead store:

```
small bowl = 150g
```

and calculate nutrition from the food’s gram-based nutrition.

For example:

```
Rice:
130 kcal / 100g
Small bowl:
150g
Calories:
```

```
130 × 150 / 100
= 195 kcal
```

This gives you flexibility later.

For example, if a user chooses:

```
250g
```

you can calculate it even if there isn’t a predefined serving.

7. Food Database Schema

foods

```
foods
---------------------------------
food id PK
name
description
category
brand
calories_per_100g
protein_per_100g
carbs_per_100g
fat_per_100g
fiber_per_100g
created_at
updated_at
```

Recommended additional fields:

8. Food Servings Table

```
food_servings
---------------------------------
serving_id PK
food_id FK -> foods.food_id
name
quantity_g
created_at
```

10. API #3 - Custom Food

Endpoint

```
POST /api/v1/diet/custom-foods
```

This is used when the food doesn’t exist in the system database.
Request

```
{
    "name": "Homemade Chicken Curry",
    "description": "Chicken curry prepared at home",
    "category": "Indian",
    "nutrition_per_100g": {
        "calories": 180,
        "protein_g": 20,
        "carbs_g": 5,
        "fat_g": 9,
        "fiber_g": 1
    }
    "is_active" : true
}
```

Response

```
{
    "food_id": 5001,
    "name": "Homemade Chicken Curry",
    "nutrition_per_100g": {
        "calories": 180,
        "protein_g": 20,
        "carbs_g": 5,
        "fat_g": 9,
        "fiber_g": 1
    }
    "is_active" : true
}
```

1. Custom Food Database

Custom foods should belong to a user.

```
custom_foods
--------------------------------
custom_food_id PK
user_id FK
```

```
name
description
category
calories_per_100g
protein_per_100g
carbs_per_100g
fat_per_100g
fiber_per_100g
created_at
updated_at
is_active
```

Every query must filter by:

```
user_id = current_user.id
```

A user must never be able to access another user’s custom foods.

12. API #4 - Get Custom Foods

Endpoint

GET /api/v1/diet/custom-foods

Query Parameters

```
search
page
page_size
```

Example:

GET /api/v1/diet/custom-foods?search=chicken

Response - only shows data for is_active : true

```
{
    "items": [
        {
            "custom_food_id": 501,
            "name": "Homemade Chicken Curry",
            "nutrition_per_100g": {
                "calories": 180,
                "protein_g": 20,
                "carbs_g": 5,
                "fat_g": 9,
```

```
                "fiber_g": 1
            }
        }
    ],
    "pagination": {
        "page": 1,
        "page_size": 20,
        "total": 1
    }
}
```

13. Custom Food Update

Endpoint

```
PATCH /api/v1/diet/custom-foods/{custom food id}
```

Example:

```
PATCH /api/v1/diet/custom-foods/501
```

Request:

```
{
    "name": "Homemade Chicken Curry",
    "nutrition_per_100g": {
        "calories": 190,
        "protein_g": 21,
        "carbs_g": 5,
        "fat_g": 9.5,
        "fiber_g": 1
    }
}
```

14. Custom Food Delete

Endpoint

DELETE /api/v1/diet/custom-foods/{custom food id}

I recommend soft deletion.

Do not physically delete the database row if that food has already been used in historical logs.

Use:

```
is_active = false
```

This preserves historical data.

15. API #5 - Saved Meals

A saved meal is a reusable collection of foods.

Example:

```
Meal: My Breakfast
2 Eggs
2 Bread
1 Banana
250ml Milk
```

16. Saved Meal Database

meals

```
meals
--------------------------------
meal_id PK
user_id FK
name
description
created_at
updated_at
```

meal_items

```
meal_item_id PK
meal_id FK
food_id nullable
custom_food_id nullable
quantity_g
serving_id nullable
created_at
```

Important constraint:

```
food_id XOR custom_food_id
```

Each meal item should refer to either:

```
system food
```

OR

```
custom food
```

but never both.

17. API - Create Saved Meal

Endpoint

```
POST /api/v1/diet/meals
```

Request

```
{
    "name": "My Breakfast",
    "items": [
        {
            "food_id": 101,
            "quantity_g": 200
        },
        {
            "food_id": 102,
            "quantity_g": 118
        },
        {
            "custom_food_id": 501,
            "quantity_g": 150
        }
```

```
    ]
}
```

The backend calculates the nutritional totals.
Response

```
{
    "meal_id": 201,
    "name": "My Breakfast",
    "items": [
        {
            "food_id": 101,
            "name": "Cooked Rice",
            "quantity_g": 200
        },
        {
            "food_id": 102,
            "name": "Banana",
            "quantity_g": 118
        },
        {
            "custom_food_id": 501,
            "name": "Homemade Chicken Curry",
            "quantity_g": 150
        }
    ],
    "nutrition": {
        "calories": 620,
        "protein_g": 32,
        "carbs_g": 72,
        "fat_g": 18
    }
}
```

18. API - Get Saved Meals

Endpoint

GET /api/v1/diet/meals

Response


```
{
  "meals": [
    {
      "meal_id": 201,
      "name": "My Breakfast",
      "nutrition": {
        "calories": 620,
        "protein_g": 32,
        "carbs_g": 72,
        "fat_g": 18
      },
      "item_count": 3,
      "items": [
        {
          "food_id": 101,
          "name": "Cooked Rice",
          "quantity_g": 200
        },
        {
          "food_id": 102,
          "name": "Banana",
          "quantity_g": 118
        }
      ]
    }
  ]
}
```


20. Meal Update

Endpoint

PATCH /api/v1/diet/meals/{meal id}

Request:

```
{
    "name": "High Protein Breakfast",
    "items": [
        {
            "food_id": 103,
            "quantity_g": 200
        },
        {
            "food_id": 102,
            "quantity_g": 118
        }
    ]
}
```

The complete meal composition can be replaced.

21. Meal Delete

Endpoint

DELETE /api/v1/diet/meals/{meal id}

Again, prefer soft deletion if necessary.

Deleting a saved meal should not delete historical food logs that were created from that meal.

22. API #6 - Log Food

This is the most important missing API from the original design.

Endpoint

POST /api/v1/diet/logs

Request

```
{
  "date": "2026-09-06",
  "meals": [
    {
      "meal_type": "Breakfast",
      "items": [
        {
          "food_id": 101,
          "quantity_g": 200
        },
        {
          "food_id": 102,
          "quantity_g": 118
        },
        {
          "custom_food_id": 501,
          "quantity_g": 150
        }
      ]
    },
    {
      "meal_type": "Lunch",
      "items": [
        {
          "food_id": 101,
          "quantity_g": 200
        },
        {
          "food_id": 102,
          "quantity_g": 118
        },
        {
          "custom_food_id": 501,
          "quantity_g": 150
        }
      ]
    }
  ]
}
```

24. Food Log Response


```
{
  "log_id": 5001,
  "date": "2026-09-06",
  "last_updated" : "2026-09-06T09:15:00Z",
  "meals": [
    {
      "meal_name": "Breakfast",
      "items": [
        {
          "food_name": "rice",
          "quantity_g": 200
        },
        {
          "food_id": "fried_rice",
          "quantity_g": 118
        },
        {
          "custom_food_id": "chicken bhuna",
          "quantity_g": 150
        }
      ],
      "nutrition": {
        "calories": 325,
        "protein_g": 6.75,
        "carbs_g": 70.5,
        "fat_g": 0.75
      }
    },
    {
      "meal_name": "Lunch",
      "items": [
        {
          "food_name": "rice",
          "quantity_g": 200
        },
        {
          "food_id": "fried_rice",
          "quantity_g": 118
        },
        {
          "custom_food_id": "chicken bhuna",
          "quantity_g": 150
        }
      ],
      "nutrition": {
        "calories": 325,
        "protein_g": 6.75,
        "carbs_g": 70.5,
        "fat_g": 0.75
```

```
      }
    }
  ]
}
```

25. Food Log Database

diet_logs

```
diet_logs
--------------------------------
log_id PK
user_id FK
date
calories
protein_g
carbs_g
fat_g
fiber_g
created_at
updated_at
```

```
logs_meals
-----------------------------------
sl_no PK
log_id FK
user_id FK
food_id nullable
custom_food_id nullable
quantity_g
```

26. Meal Types

Use an enum.

```
BREAKFAST
LUNCH
DINNER
SNACK
OTHER
```

Database value:

```
breakfast
lunch
dinner
snack
other
```

Do not hardcode arbitrary strings throughout the backend.

29. API - Get Daily Food Logs

Endpoint

```
GET /api/v1/diet/logs?date=2026-09-06
```

Response


```
{
  "log_id": 5001,
  "date": "2026-09-06",
  "last_updated": "2026-09-06T09:15:00Z",
  "meals": [
    {
      "meal_name": "Breakfast",
      "items": [
        {
          "food_name": "rice",
          "quantity_g": 200,
          "calories": 143,
          "protein_g": 12.6,
          "carbs_g": 1.1,
          "fat_g": 9.5
        },
        {
          "food_id": "fried_rice",
          "quantity_g": 118,
          "calories": 143,
          "protein_g": 12.6,
          "carbs_g": 1.1,
```

```
          "fat_g": 9.5
        },
        {
          "custom_food_id": "chicken bhuna",
          "quantity_g": 150,
          "calories": 143,
          "protein_g": 12.6,
          "carbs_g": 1.1,
          "fat_g": 9.5
        }
      ],
      "nutrition": {
        "calories": 325,
        "protein_g": 6.75,
        "carbs_g": 70.5,
        "fat_g": 0.75
      }
    },
    {
      "meal_name": "Lunch",
      "items": [
        {
          "food_name": "rice",
          "quantity_g": 200,
          "calories": 143,
          "protein_g": 12.6,
          "carbs_g": 1.1,
          "fat_g": 9.5
        },
        {
          "food_id": "fried_rice",
          "quantity_g": 118,
          "calories": 143,
          "protein_g": 12.6,
          "carbs_g": 1.1,
          "fat_g": 9.5
        },
        {
```

```
          "custom_food_id": "chicken bhuna",
          "quantity_g": 150,
          "calories": 143,
          "protein_g": 12.6,
          "carbs_g": 1.1,
          "fat_g": 9.5
        }
      ],
      "nutrition": {
        "calories": 325,
        "protein_g": 6.75,
        "carbs_g": 70.5,
        "fat_g": 0.75
      }
    }
  ]
}
```

30. API - Edit Food Log

Endpoint

PATCH /api/v1/diet/logs/{log id}

Request:

```
{
  "date": "2026-09-06",
  "meals": [
    {
      "meal_type": "Breakfast",
      "items": [
        {
          "food_id": 101,
          "quantity_g": 300
        },
        {
          "food_id": 102,
          "quantity_g": 118
```

```
        },
        {
          "custom_food_id": 501,
          "quantity_g": 150
        }
      ]
    },
    {
      "meal_type": "Lunch",
      "items": [
        {
          "food_id": 101,
          "quantity_g": 200
        },
        {
          "food_id": 102,
          "quantity_g": 118
        },
        {
          "custom_food_id": 501,
          "quantity_g": 150
        }
      ]
    }
  ]
}
```

Backend recalculates:

```
calories
protein
carbs
fat
fiber
```

the request sturcture is same as the POST that is while creating the log
the backedn will compare it with the DB is any change is this then it will update accordingly.

31. API - Delete Food Log

Endpoint

DELETE /api/v1/diet/logs/{log id}

Response:

204 No Content

32. API - Diet History

This is useful for both the frontend and future AI coach.
Endpoint

GET /api/v1/diet/history

Query:

```
start_date
end_date
```

Example:

GET /api/v1/diet/history?start date=2026-09-01&end date=2026-09-07

Response:

```
{
    "start_date": "2026-09-01",
    "end_date": "2026-09-07",
    "days": [
        {
            "date": "2026-09-01",
            "calories": 2450,
            "protein_g": 165,
            "carbs_g": 280,
            "fat_g": 72
            },
        {
            "date": "2026-09-02",
            "calories": 2510,
            "protein_g": 172,
            "carbs_g": 290,
            "fat_g": 75
        }
    ],
    "average": {
        "calories": 2480,
        "protein_g": 168,
```

```
        "carbs_g": 285,
        "fat_g": 73
    }
}
```

33. API - Nutrition Goals

The calorie budget and macro targets should come from the user's diet goals.
Get goals

```
GET /api/v1/diet/goals
```

Response:

```
{
    "calories": 2500,
    "protein_g": 170,
    "carbs_g": 280,
    "fat_g": 75
}
```

34. Update Goals

Endpoint

```
PUT /api/v1/diet/goals
```

Request:

```
{
    "calories": 2500,
    "protein_g": 170,
    "carbs_g": 280,
    "fat_g": 75
}
```

35. Diet Goals Database

diet_goals

```
diet_goals
------------------------------
goal_id PK
user_id FK UNIQUE
calorie_target
protein_target_g
carbs_target_g
fat_target_g
created_at
updated_at
```

A user should have one active goal configuration.

Later, if required, this can be extended to support historical goals.

41. Authentication

All user-specific APIs should use the existing LiftLog authentication system.

For example:

```
Authorization: Bearer <token>
```

The backend should obtain:

```
user_id
```

from the authenticated token.